

# Your SaaS Can Work and Still Be Unsafe.

I help founders and engineering teams find and fix the production risks that rapid development and AI-assisted coding often leave behind - before those risks become customer incidents.

|                        |                            |                         |                                         |
|------------------------|----------------------------|-------------------------|-----------------------------------------|
| 4 / 4                  | 401 / 400 / 429            | 0                       | SELECT only                             |
| Security tests passing | API failure paths verified | Anonymous DB privileges | Authenticated client DB write privilege |

**What I do.** I review fast-built SaaS products, identify the hidden trust-boundary and production-readiness problems, then implement the fixes directly in the codebase. The goal is not a 60-page audit. The goal is a safer product that can actually ship.

| If this sounds familiar                                                        | I can help with                                                |
|--------------------------------------------------------------------------------|----------------------------------------------------------------|
| Built quickly with Cursor, Lovable, Replit, Bolt, Claude Code, or a freelancer | Production-readiness review + hands-on remediation             |
| App works, but you are not sure whether auth / RLS / API boundaries are safe   | Authentication, authorization, tenant isolation, API hardening |
| You are about to onboard real users or paying customers                        | Risk reduction before production or customer pilots            |

**Positioning:** I do not sell vague “AI consulting.” I find concrete production risks, fix them, and prove the result.

**Proof:** This document shows one complete rescue with inspectable code, database changes, screenshots, tests, and before/after Git history.

# 1. What I Found - and What I Fixed

The demo product looked usable on the surface. Underneath, it had several patterns that would make me uncomfortable putting real customer data or paid AI usage through it.

| Production risk   | BEFORE                              | AFTER MY RESCUE                                |
|-------------------|-------------------------------------|------------------------------------------------|
| Authentication    | No application login                | Supabase Auth + bearer-token API verification  |
| Tenant isolation  | Cross-workspace visibility possible | Database-enforced RLS via workspace membership |
| Browser DB writes | Anonymous/direct client write path  | Privileged writes moved server-side            |
| AI/API abuse      | No validation or rate limit         | Validation + per-user rate limiting            |
| DB permissions    | Broad client-facing privileges      | anon: none; authenticated: SELECT only         |
| Verification      | No security-path tests              | Automated tests + lint + production build      |

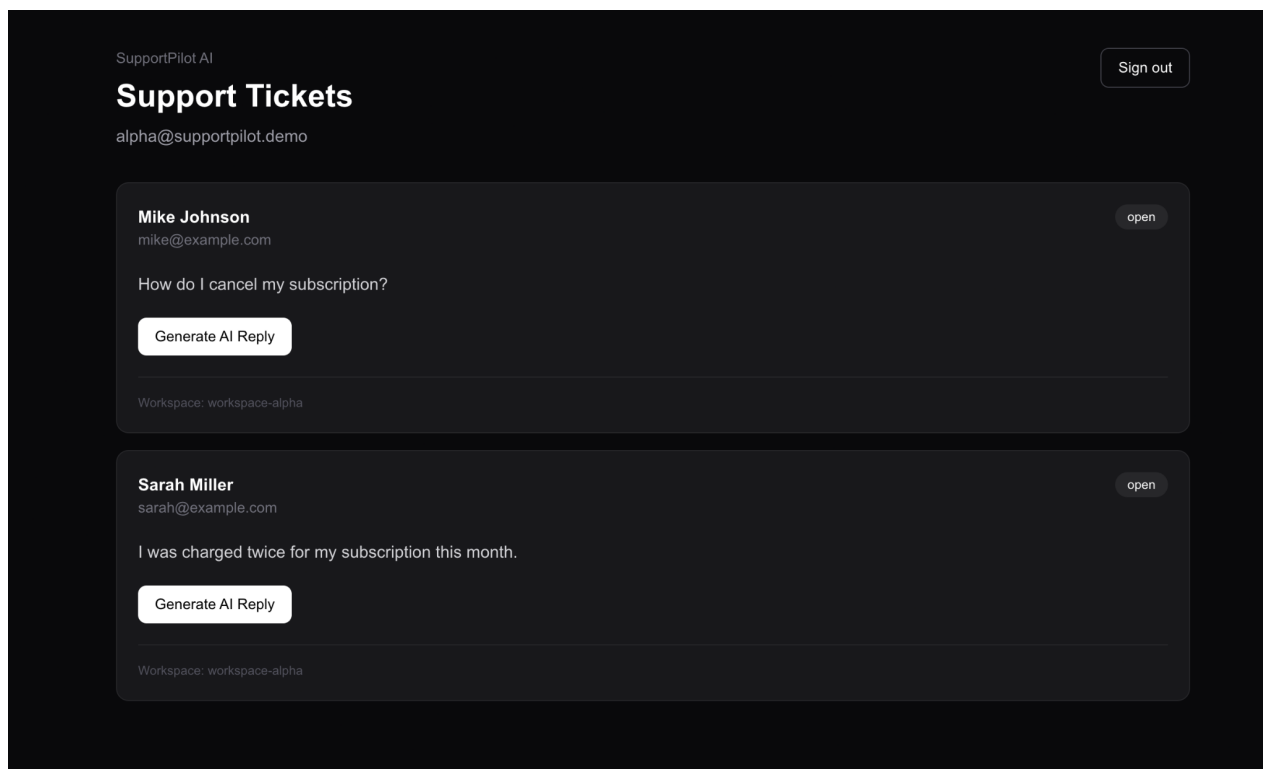
## Why clients hire me for this

- **I work past the UI.** A polished dashboard does not tell you whether the trust boundaries are safe.
- **I fix the problem, not just describe it.** The deliverable is code, policies, migrations, tests, and a working flow.
- **I preserve velocity.** The point is to harden the product without turning an MVP into an enterprise rewrite.
- **I leave evidence.** Your team can inspect exactly what changed and why.

**Client outcome: clearer access boundaries, less accidental exposure, controlled AI/API usage, and a codebase that is easier to trust before real customers arrive.**

## 2. Case Study: Tenant Isolation That Actually Holds

One of the highest-risk issues in multi-tenant SaaS is accidental cross-customer visibility. I moved the authorization boundary into PostgreSQL Row Level Security so access is enforced by the database, not by frontend filtering.



Evidence: the authenticated workspace-alpha user sees only Sarah Miller and Mike Johnson. The workspace-beta ticket is not returned.

**What this means for your product: I verify who is allowed to see what, then enforce that rule where bypassing the UI cannot defeat it.**

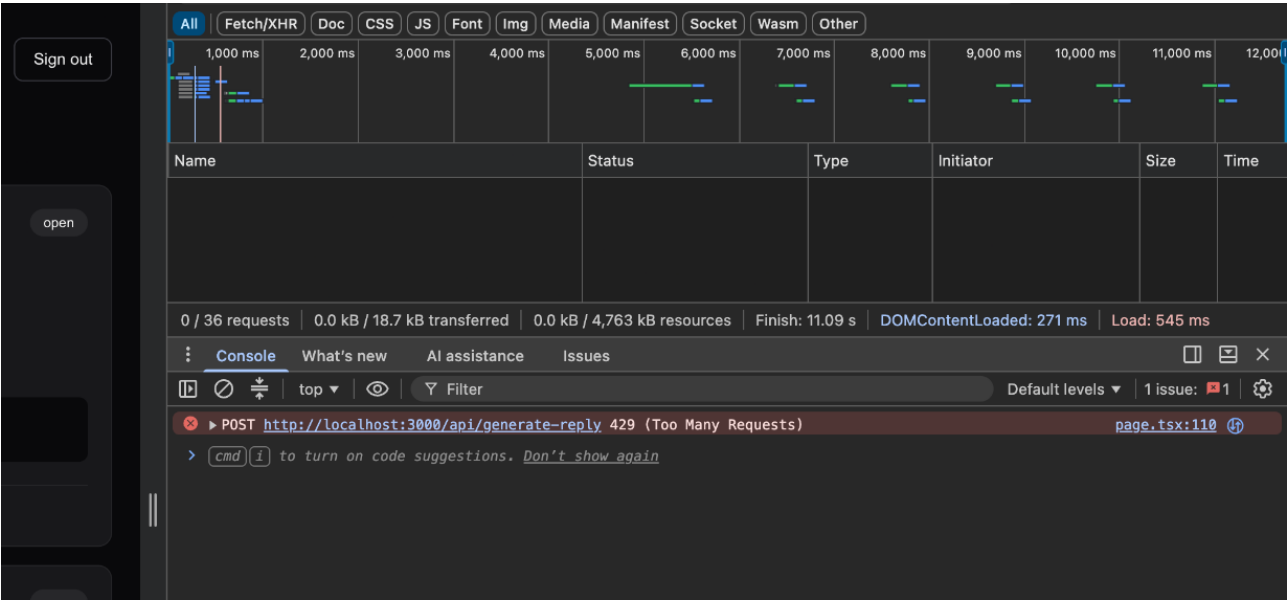
### What I would inspect in a real client codebase

- Tenant IDs trusted from the browser instead of the authenticated identity.
- RLS disabled, incomplete, or inconsistent across tables.
- Authorization checks that exist only in React components or route guards.
- Service-role / admin keys leaking into code paths that should be user-scoped.

### 3. Case Study: Secure the Expensive Endpoint

AI endpoints are easy to abuse because every request may represent real cost. I hardened the reply-generation route so requests must be authenticated, valid, authorized, and within a defined usage window.

| Control        | Implemented behavior                                     |
|----------------|----------------------------------------------------------|
| Authentication | Missing/invalid token → <b>401 Unauthorized</b>          |
| Validation     | Invalid request → <b>400 Bad Request</b>                 |
| Rate limiting  | Excess usage → <b>429 Too Many Requests</b>              |
| Authorization  | Ticket access checked through the user-scoped RLS client |
| Write boundary | Privileged database update remains server-only           |



Evidence: repeated calls trigger HTTP 429. The limiter is PostgreSQL-backed, so the protection is not tied to one application process.

**Client value: authenticated endpoints, controlled usage, and fewer ways for an exposed API to become an unexpected bill or operational incident.**

## 4. Case Study: Reduce What the Browser Is Allowed to Do

After moving sensitive writes server-side, I reduced the database privileges exposed to application clients. This is defense-in-depth: RLS limits which rows a user can see, while grants limit which operations that role can perform at all.

| Role                         | Final privilege model                               |
|------------------------------|-----------------------------------------------------|
| anon                         | No table privileges                                 |
| authenticated                | <b>SELECT only</b> - still constrained by RLS       |
| service_role / server secret | Privileged operations reserved for server-side code |

```
7 and grantee in ('anon', 'authenticated', 'service_role')
8 order by grantee, privilege_type;
```

| Results Chart |                |
|---------------|----------------|
| grantee       | privilege_type |
| authenticated | SELECT         |
| service_role  | DELETE         |
| service_role  | INSERT         |
| service_role  | REFERENCES     |
| service_role  | SELECT         |
| service_role  | TRIGGER        |
| service_role  | TRUNCATE       |
| service_role  | UPDATE         |

Evidence: authenticated application clients retain SELECT only; privileged operations remain available to the server role.

**What I bring: I do not rely on one security control. I prefer layered guardrails so one mistake does not automatically become a production incident.**

## 5. Proof, Deliverables, and How I Can Help

The rescue finishes with executable proof. That matters because production hardening should be reviewable by another engineer, not dependent on trust in a consultant's slide deck.

```
$ npx vitest run
Test Files 1 passed (1)
Tests 4 passed (4)

PASS 401 - missing authentication
PASS 400 - invalid request
PASS 429 - rate limit exceeded
PASS 200 - authorized request
```

| What you get from me        | Typical output                                                       |
|-----------------------------|----------------------------------------------------------------------|
| Production-readiness review | Prioritized findings with concrete risk and remediation path         |
| Hands-on hardening          | Actual code, policies, migrations, and API fixes - not just advice   |
| Verification                | Tests, build checks, before/after evidence, and reproducible changes |
| Handoff                     | Clear PR history and technical notes your team can continue from     |

### If your SaaS was built fast, I can tell you what is still risky before your customers do.

Best fit: AI-built SaaS, rapid MVPs, Supabase/Next.js products, inherited codebases, and products preparing for their first real customer rollout.

**Start with the code:** [github.com/ivandj0h/ai-saas-production-rescue-demo](https://github.com/ivandj0h/ai-saas-production-rescue-demo)  
**Before/After:** [github.com/ivandj0h/ai-saas-production-rescue-demo/compare/before-rescue...after-rescue](https://github.com/ivandj0h/ai-saas-production-rescue-demo/compare/before-rescue...after-rescue)

If you received this PDF directly from me, reply with your repo/stack and the part of production that worries you most.